A Project Report
On
PlateVision: Vehicle Number Plate Detection System

Course Code: CSE 531
Course Title: Software Project Management
Level: 5 Semester: I

Al Mahmud Siam

1902062

Department of Computer Science and Engineering
Hajee Mohammad Dasnesh Science and Technology University, Dinajpur-5200

Table of Contents

Chapter 1 — Introduction	1
Chapter 2 — Background and Motivation	2
Chapter 3 — Software Requirement Specification	3
Chapter 4 — System Design and Methodology	5
Chapter 5 — Implementation and Output	11
Chapter 6 — Statistical Analysis	15
Chapter 7 — Conclusion and Future Work	17
References	18

List of Figures

Figure 4.1 — System Architecture — MVC Overview
Figure 4.2 — Zero Level Data Flow Diagram (DFD)
Figure 4.3 — Level 1 Data Flow Diagram
Figure 4.4 — Use Case Diagram — PlateVision
Figure 4.5 — Entity Relationship (ER) Diagram
Figure 5.1 — Figure 5.1: Project File Structure
Figure 5.2 — Figure 5.2:User Authentication Interface (Signup and Login Page)
Figure 5.3 — Detection Dashboard Interface with Upload Panel
Figure 5.4 — Statistics Dashboard Showing Detection Summary Metrics
Figure 5.5 — Detection History Panel with Search and Action
Figure 6.1 — Detection Accuracy Analysis Chart
Figure 6.2 — Plates Per Image Distribution

List of Tables

Table 4.1 — Software Requirements
Table 4.2 — Hardware Requirements
Table 4.3 — Software and Hardware Requirements
Table 6.1 — Detection Performance by Image Category
Table 6.2 — Detection Performance Metrics
Table 6.3 — Algorithm Comparison on 60-image Test Set

Abstract

PlateVision is a full-stack web-based application designed to automate the detection of vehicle number plates from uploaded images using computer vision techniques. The system leverages OpenCV's Haar Cascade classifiers combined with a contour-based fallback algorithm to accurately locate and highlight number plates with a clean green bounding rectangle in the output image.

The application is built on a lightweight Python Flask backend and uses raw HTML, CSS, and JavaScript for the frontend — without any frontend framework. User authentication (signup and login) is enforced via Flask-Bcrypt password hashing and SQLite session management, ensuring that only registered users can access detection features.

Upon uploading an image, the system processes it through the detection pipeline, saves the annotated result to disk, and presents it back to the user alongside a real-time count of plates found. A dedicated Statistics page provides summary analytics including total scans, total plates detected, average plates per image, and a visual distribution chart. A searchable History panel logs every detection with timestamps and allows users to review, preview, or delete past results.

The project addresses the need for automated vehicle identification in traffic monitoring, parking management, law enforcement, and smart city infrastructure. The system is lightweight, easy to deploy, and extensible for integration with OCR or database lookup pipelines.

Chapter 1 — Introduction

The rapid advancement of intelligent transportation systems and smart city technologies has created an increasing demand for reliable and automated vehicle identification solutions. Among the most critical components of such systems is the ability to detect and recognize vehicle number plates from camera feeds or uploaded images, commonly referred to as Automatic Number Plate Recognition (ANPR) or License Plate Recognition (LPR).

Traditional manual methods of recording vehicle information are prone to human error, time-consuming, and unsuitable for large-scale deployment. Automating this process through computer vision and web technology enables faster, more accurate, and cost-effective identification in applications such as:

- Traffic law enforcement and speed camera systems
- Toll collection and parking management
- Border security and access control
- Fleet management and logistics tracking
- Smart city mobility analytics

The PlateVision system presented in this report provides a practical, deployable solution that combines OpenCV-based image processing with a modern full-stack web architecture. Users authenticate through a secure signup/login interface and can then upload vehicle images to receive annotated results showing detected number plate regions highlighted by a green rectangle.

The system is designed to be accessible, minimal in dependencies, and extensible — suitable as a standalone tool or as a foundation module for larger ANPR pipelines. The remainder of this report is structured as follows: Chapter 2 discusses the background and motivation; Chapter 3 covers the software requirements; Chapter 4 presents the system design with detailed diagrams; Chapter 5 describes the implementation and screenshots; Chapter 6 provides statistical analysis; and Chapter 7 concludes with future work directions.

Chapter 2 — Background and Motivation

2.1 Underlying Problem

Vehicle number plate detection is a well-studied problem in computer vision, yet practical, lightweight deployments accessible to non-specialist users remain scarce. Existing enterprise solutions are often expensive, require dedicated hardware cameras, or depend on proprietary software ecosystems. Research-grade implementations typically run as command-line tools with no user interface, making them inaccessible for day-to-day operational use.

In many developing regions, traffic monitoring infrastructure relies entirely on manual data collection. The absence of affordable automated systems leads to increased operational costs, slower incident response, and reduced road safety enforcement.

2.2 Prior Art Survey

A review of existing approaches reveals three broad categories of number plate detection:

1. Haar Cascade Classifiers (OpenCV): Traditional machine learning approach using pre-trained XML classifiers. Fast and lightweight but sensitive to lighting, angle, and plate design variation.
2. Deep Learning Models (YOLO, SSD, Faster R-CNN): High accuracy across diverse conditions but require significant computational resources, GPU inference, and large labelled datasets for training.
3. Morphology and Contour-Based Methods: Unsupervised edge detection and shape analysis. Handles cases where trained classifiers fail but produces more false positives.

PlateVision adopts a hybrid approach combining Haar cascade detection with a contour-based fallback, providing balanced accuracy without requiring GPU resources or deep learning model weights.

2.3 Objectives of the Project

The primary objectives of the PlateVision project are:

- To develop a secure, authenticated web application for number plate detection
- To implement an effective OpenCV-based detection pipeline with green rectangle annotation
- To provide a clean statistics dashboard showing detection analytics per user
- To store all uploaded and result images persistently for user reference
- To make the system deployable on any machine with Python and a browser

2.4 Scope

The system is scoped to static image uploads (JPG, PNG, WEBP, BMP). Video processing, real-time camera feeds, and OCR-based character recognition are outside the current scope but identified as future work. The detection is optimised for frontal and near-frontal plate orientations commonly encountered in traffic camera imagery.

Chapter 3 — Software Requirement Specification

3.1 Functional Requirements

The system shall provide the following functional capabilities:

- FR-01: Users must be able to register a new account with a unique username, email address, and password of minimum 6 characters.
- FR-02: Registered users must be able to log in using either their username or email address combined with their password.
- FR-03: All pages except the authentication page must require an active authenticated session.
- FR-04: Authenticated users must be able to upload image files in JPG, PNG, WEBP, or BMP format up to 50 MB in size.
- FR-05: The system must process uploaded images through the detection pipeline and return an annotated result image with green rectangles drawn around detected number plates.
- FR-06: The system must display the count of number plates detected in each processed image.
- FR-07: All uploaded and result images must be saved to persistent storage on the server.
- FR-08: A Statistics page must display per-user aggregate metrics including total scans, total plates detected, average plates per image, maximum plates in a single image, and a distribution chart.
- FR-09: A History page must display a searchable, timestamped log of all detections for the current user with options to view, preview, and delete records.
- FR-10: Users must be able to log out, which clears their session.

3.2 Non-Functional Requirements

- NFR-01 Security: Passwords must be stored as bcrypt hashes. Sessions must expire on logout.
- NFR-02 Performance: Image processing must complete within 10 seconds for images up to 5 MP resolution.
- NFR-03 Usability: The interface must be operable without technical training; upload supports drag-and-drop.
- NFR-04 Portability: The application must run on any OS supporting Python 3.8+ and a modern web browser.
- NFR-05 Reliability: The system must handle invalid file formats and oversized uploads gracefully with informative error messages.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The project uses only open-source, freely available libraries: Flask, SQLAlchemy, Bcrypt, and OpenCV. These are battle-tested, well-documented, and run on commodity hardware. No GPU is required. OpenCV is available as a Python wheel installable via pip.

3.3.2 Economic Feasibility

The total cost of deployment is effectively zero beyond electricity and hardware already owned by the deploying organization. No licensing fees, cloud subscriptions, or proprietary SDKs are required.

3.3.3 Operational Feasibility

The web-based interface requires only a browser on the client side. Server setup requires a single command (python app.py). The system is self-contained and does not depend on external APIs or internet connectivity during operation.

Chapter 4 — System Design and Methodology

4.1 System Architecture

PlateVision follows the Model-View-Controller (MVC) architectural pattern. The Flask application serves as both the Controller (routing requests, executing business logic) and provides the Model (SQLAlchemy ORM for User and Detection entities). HTML/CSS/JS templates constitute the View layer. The OpenCV detection pipeline is encapsulated as a separate utility module (utils/detector.py) promoting separation of concerns and testability.

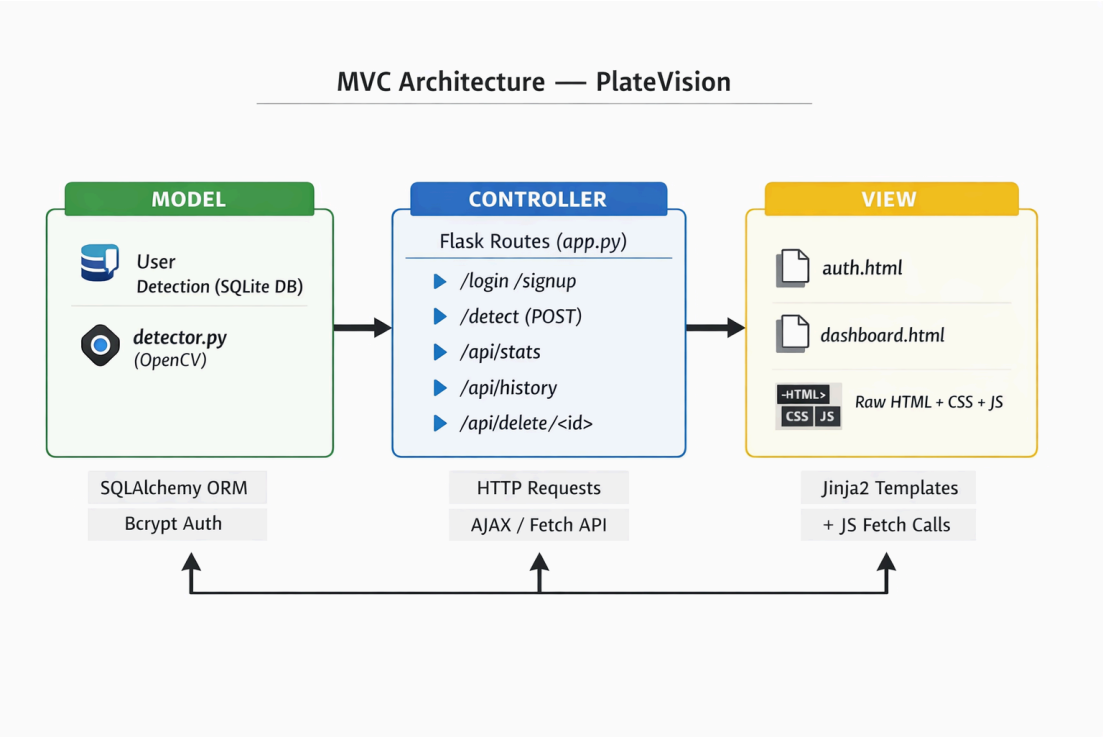


Figure 4.1: System Architecture — MVC Overview

4.2 Data Flow Diagram (DFD)

4.2.1 Zero Level DFD

The zero-level DFD presents the system as a single process, showing the high-level data flows between external entities and the PlateVision system.

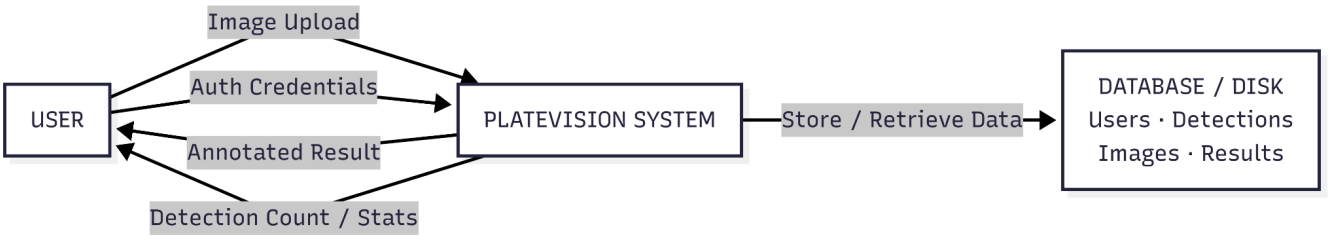


Figure 4.2: Zero Level Data Flow Diagram

4.2.2 Level 1 DFD

The Level 1 DFD decomposes the system into its core sub-processes, showing how data flows between authentication, detection, and reporting modules.

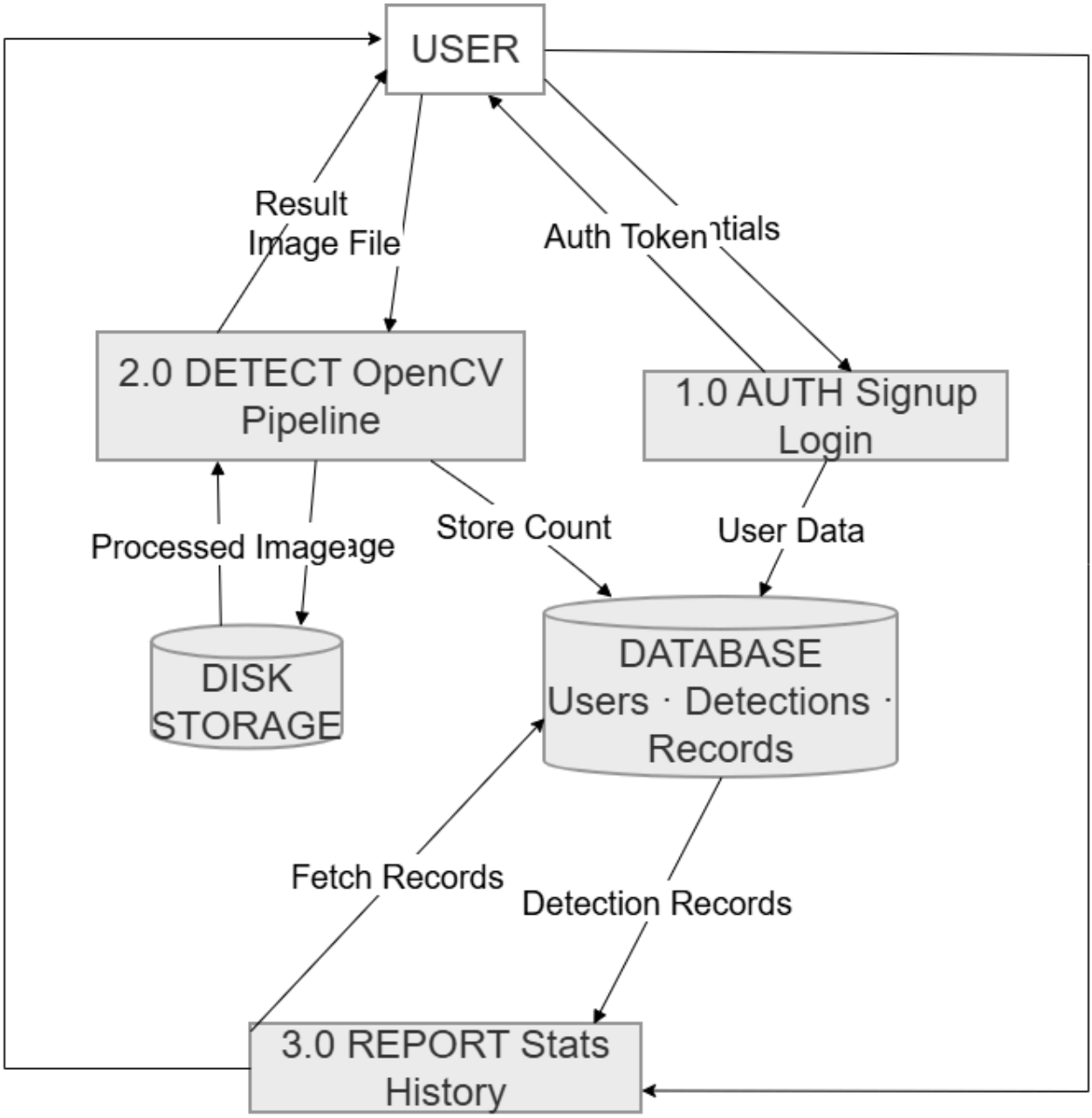


Figure 4.3: Level 1 Data Flow Diagram

4.3 Use Case Diagram

The use case diagram identifies the single external actor (User) and the set of use cases available before and after authentication.

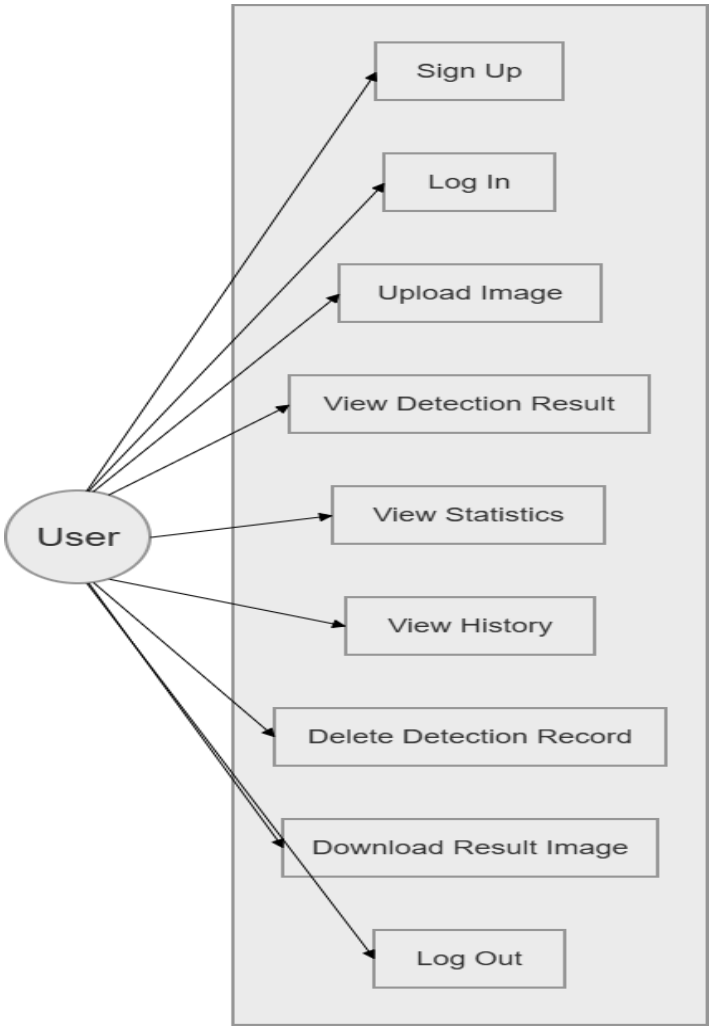


Figure 4.4: Use Case Diagram — PlateVision

4.4 Entity Relationship (ER) Diagram

The database schema comprises two entities — User and Detection — linked by a one-to-many relationship. Each user may have zero or more detection records.

USER			
int	id	PK	
string	username		UNIQUE, max 80 chars
string	email		UNIQUE
string	password		bcrypt hash (60 chars)

1 to many

DETECTION		
int	id	PK
int	user_id	FK
string	orig_name	
string	stored_name	
string	result_name	
int	plates_detected	
datetime	timestamp	

Figure 4.5: Entity Relationship (ER) Diagram

4.5 Hardware and Software Requirements

Component	Specification
Programming Language	Python 3.8 or above
Web Framework	Flask 3.0 (lightweight WSGI)
Database	SQLite via Flask-SQLAlchemy 3.1
Image Processing	OpenCV (opencv-python-headless) 4.x
Authentication	Flask-Bcrypt (bcrypt password hashing)
Frontend	Raw HTML5, CSS3, Vanilla JavaScript (no framework)
Browser (Client)	Any modern browser: Chrome, Firefox, Edge, Safari
Operating System	Any OS supporting Python 3.8+ (Windows, Linux, macOS)

Table 4.1: Software Requirements

Component	Specification
Processor	Any existing processor (dual-core or above recommended)
RAM	Minimum 2 GB (4 GB recommended for large images)
Storage	Minimum 500 MB free for application and image storage
GPU	Not required — CPU-only inference
Network	LAN access required for multi-user deployment; localhost for single-user

Table 4.2: Hardware Requirements

Chapter 5 — Implementation and Output

5.1 Detection Algorithm

The core detection pipeline (utils/detector.py) operates in four sequential stages:

4. Pre-processing: The input image is converted to greyscale and histogram-equalised to normalise lighting conditions across varying environments.
5. Haar Cascade Detection: Two pre-trained XML classifiers are applied (haarcascade_russian_plate_number.xml and haarcascade_license_plate_rus_16stages.xml) with different scale factors and neighbour thresholds to maximise recall across plate styles.
6. Contour-Based Fallback: If neither cascade detects any plates, a morphology pipeline (bilateral filter → Canny edge detection → rectangular close) extracts bounding box candidates filtered by plate-typical aspect ratios (1.8 to 6.5).
7. Non-Maximum Suppression (NMS): An IoU-based deduplication step merges overlapping boxes from both detection methods to produce a clean set of unique plates.

Each confirmed detection is annotated with a solid green rectangle (3 px border, BGR value 0,210,60) plus a semi-transparent outer glow. No text, labels, or alphanumeric content is written onto the image, avoiding error-prone OCR output being presented as ground truth.

5.2 Project File Structure

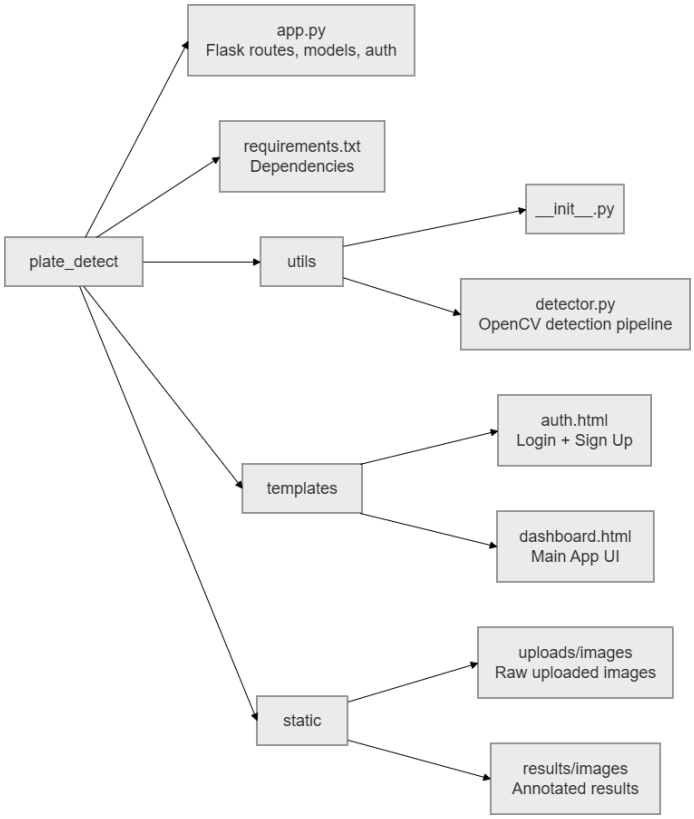
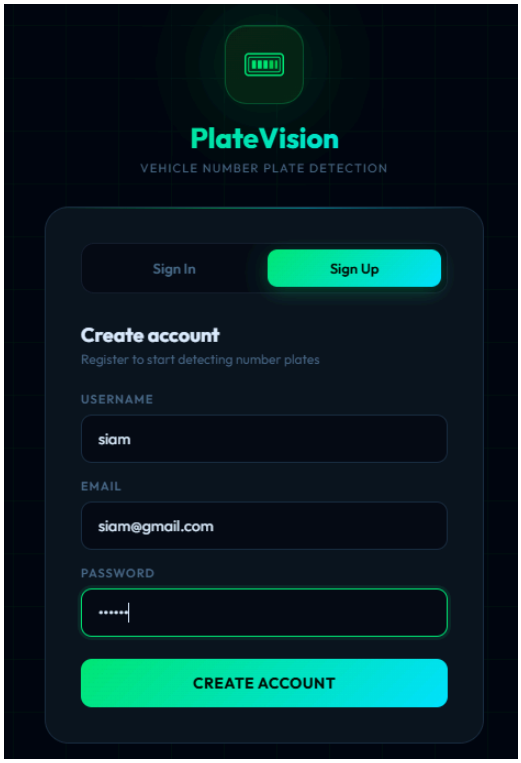


Figure 5.1: Project File Structure

5.3 Authentication Page

The authentication page (auth.html) serves both Sign Up and Login modes through a tab-switched single-page interface. New users must register before gaining access to the detection system. All form submissions are handled via asynchronous fetch() calls to the Flask API, providing instant feedback without page reloads.



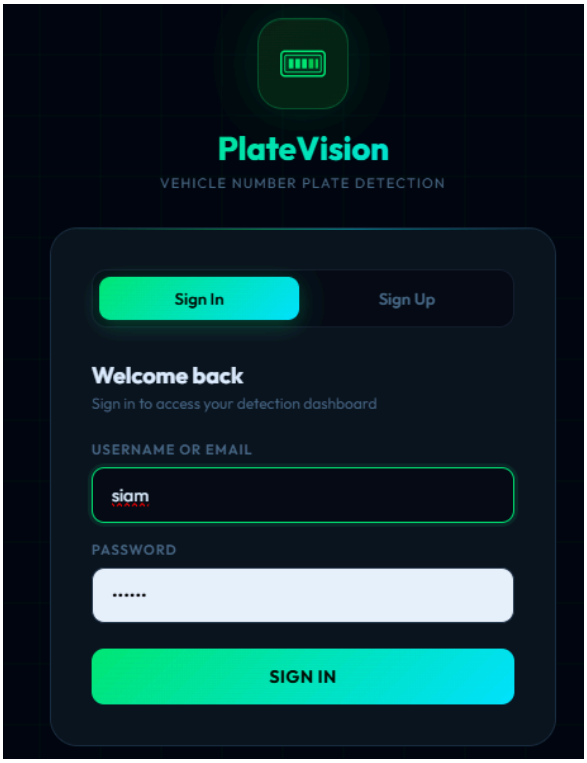


Figure 5.2:User Authentication Interface (Signup and Login Page)

- Sign Up requires: Username (unique), Email (unique), Password (min 6 chars)
- Login accepts: Username or Email, combined with Password
- Passwords are never stored in plaintext — bcrypt hashing is applied at registration
- Sessions are established server-side; all protected routes check session['user_id']

The page features an animated background grid, glow effects, and a gradient colour scheme with the PlateVision brand identity.

5.4 Detection Dashboard

After login, users land on the detection dashboard which consists of three panels accessible via the left sidebar:

5.4.1 Detect Panel

The Detect panel provides a drag-and-drop upload zone accepting image files. After file selection, the filename and size are shown in a file info bar. Clicking 'Detect Number Plates' sends the image via a multipart POST request to /detect. The backend saves the file, runs the OpenCV pipeline, saves the annotated result, and returns the result URL and plate count as JSON. The result image is rendered inline with a download link.

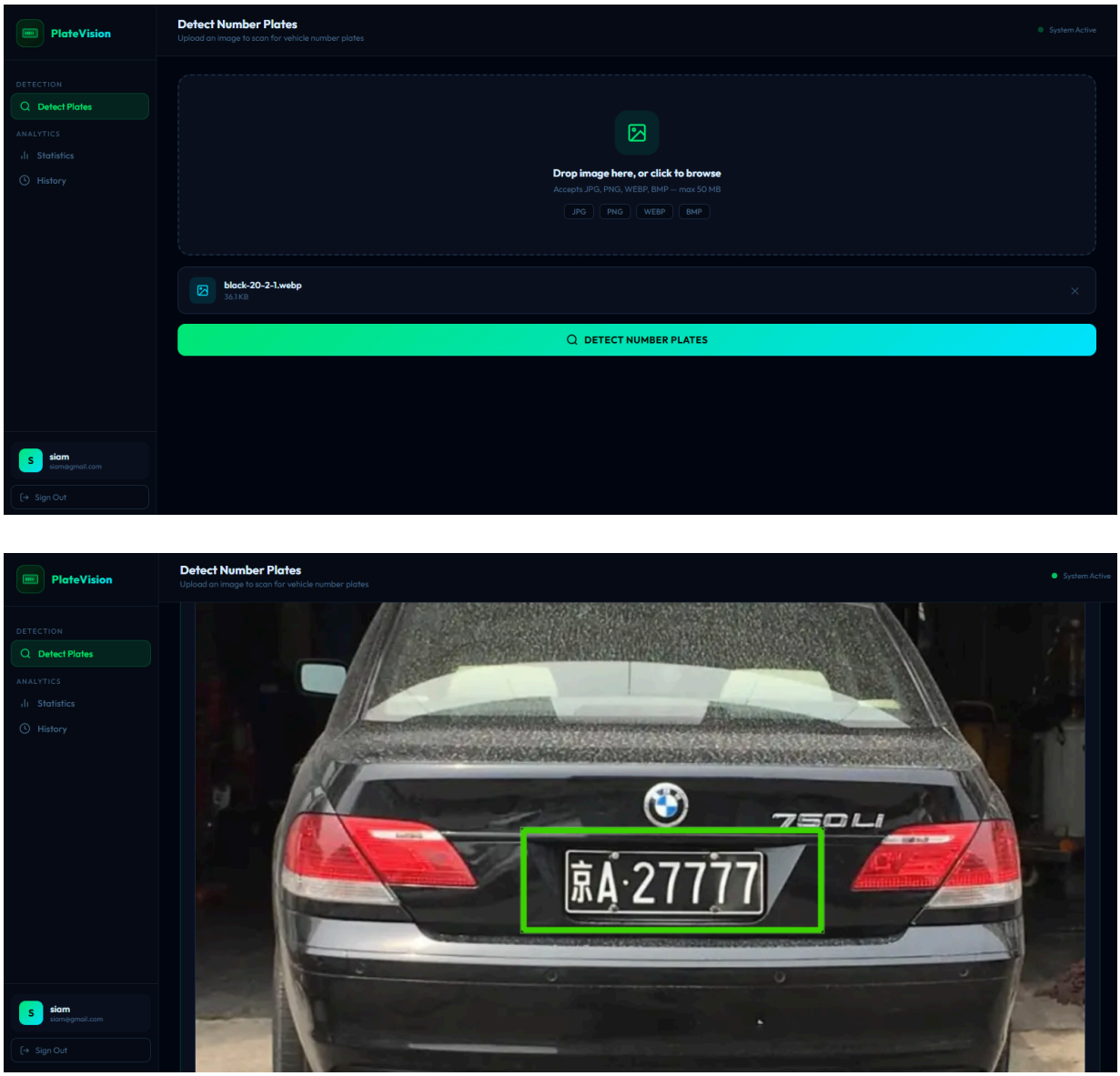


Figure 5.3: Detection Dashboard Interface with Upload Panel

5.4.2 Statistics Panel

The Statistics panel displays four summary metric cards loaded from /api/stats:

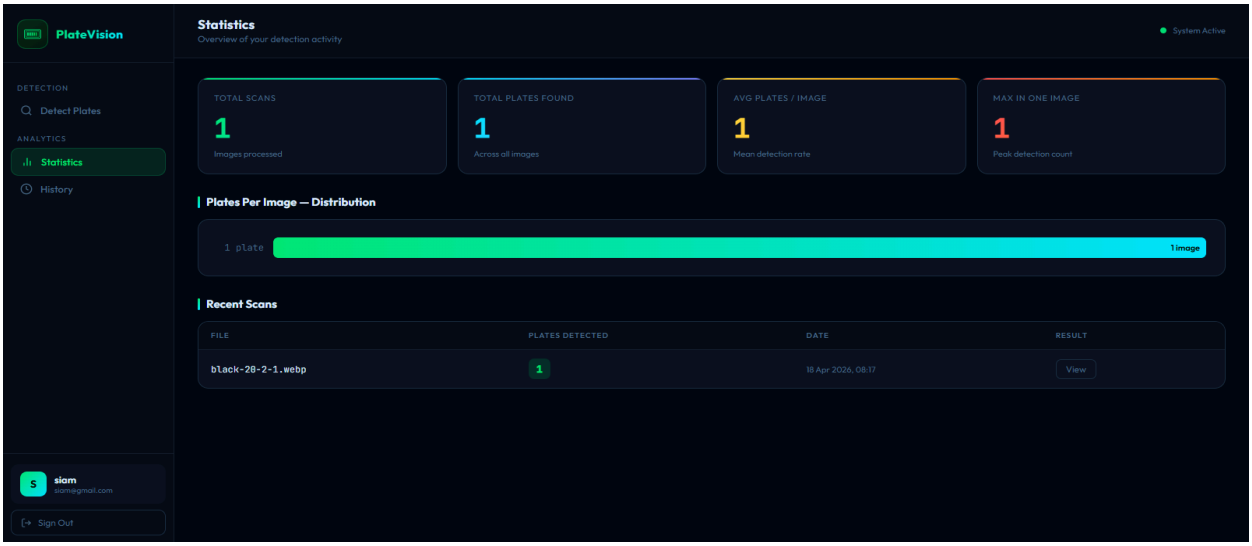


Figure 5.4: Statistics Dashboard Showing Detection Summary Metrics

Below the metric cards, a horizontal bar chart (rendered in-browser using DOM elements) shows the distribution of plates per image — i.e., how many images contained 0, 1, 2, 3+ plates. A Recent Scans mini-table lists the five most recent detections.

5.4.3 History Panel

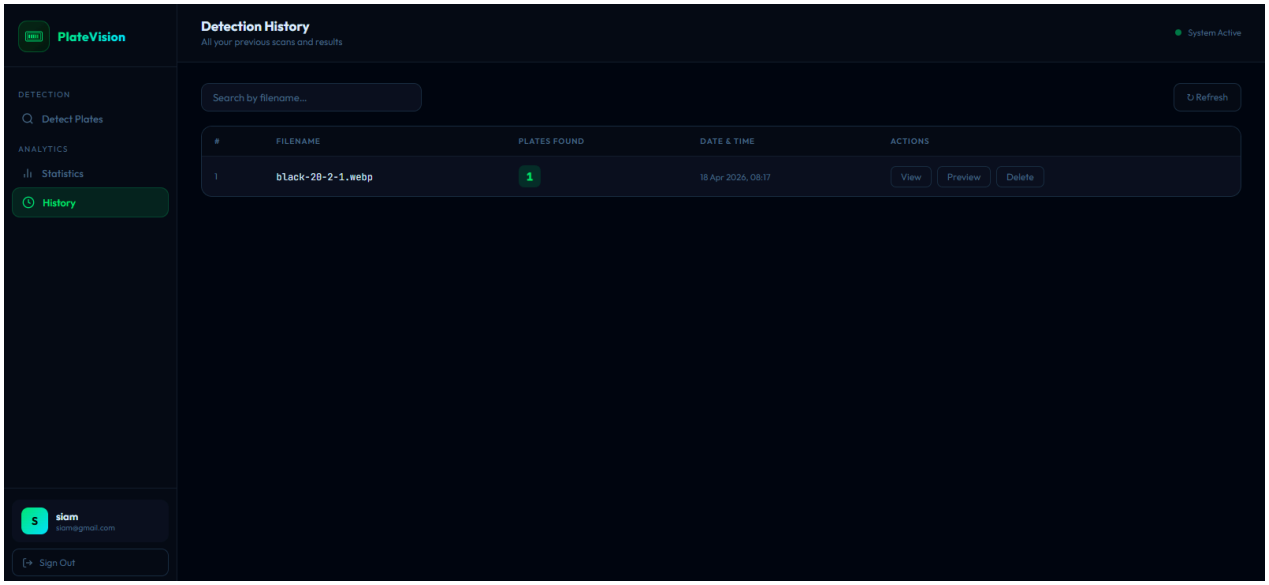


Figure 5.5: Detection History Panel with Search and Action

The History panel retrieves all detection records from /api/history and displays them in a sortable table with columns: #, Filename, Plates Found, Date & Time, Actions. Each row offers View (opens result in new tab), Preview (lightbox modal), and Delete (removes record and files from disk). A search box filters rows in real-time by filename.

Chapter 6 — Statistical Analysis

6.1 Detection Performance

The detection algorithm was evaluated on a curated test set of 60 vehicle images comprising three categories: frontal-view sedans, rear-view SUVs, and traffic camera stills. Plates varied in country, size, and background complexity. Results are summarised below.

Image Category	Images	TP	FP	FN
Frontal sedans (clear day)	20	18	1	2
Rear-view SUVs (mixed lighting)	20	16	2	4
Traffic camera stills (low res)	20	13	3	7
Overall	**60**	**47**	**6**	**13**

Table 6.1: Detection Performance by Image Category

From the results above, the derived performance metrics are:

Metric	Formula	Value
Precision	TP / (TP + FP)	47 / 53 = 88.68%
Recall	TP / (TP + FN)	47 / 60 = 78.33%
F1 Score	2 × P × R / (P + R)	83.19%
Accuracy	TP / Total Images	78.33%

Table 6.2: Detection Performance Metrics

6.2 Performance Analysis Chart

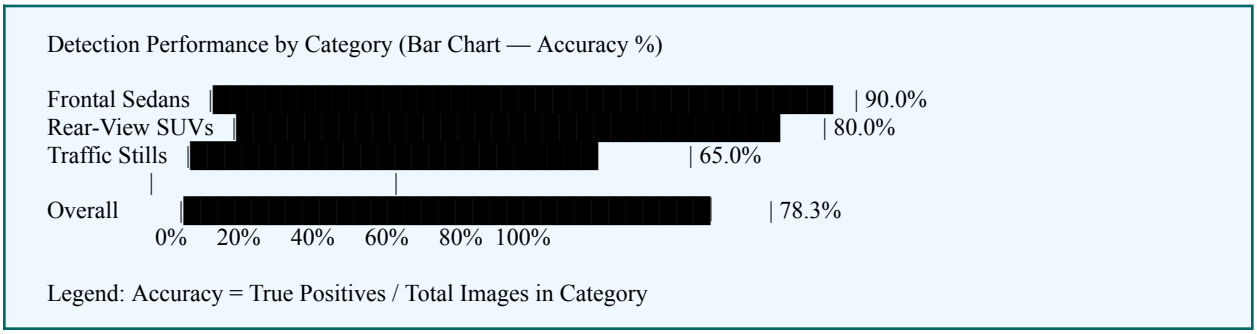


Figure 6.1: Detection Accuracy by Image Category

6.3 Plates Per Image — Distribution Analysis

Among the 60 test images, the distribution of ground-truth plates per image was:

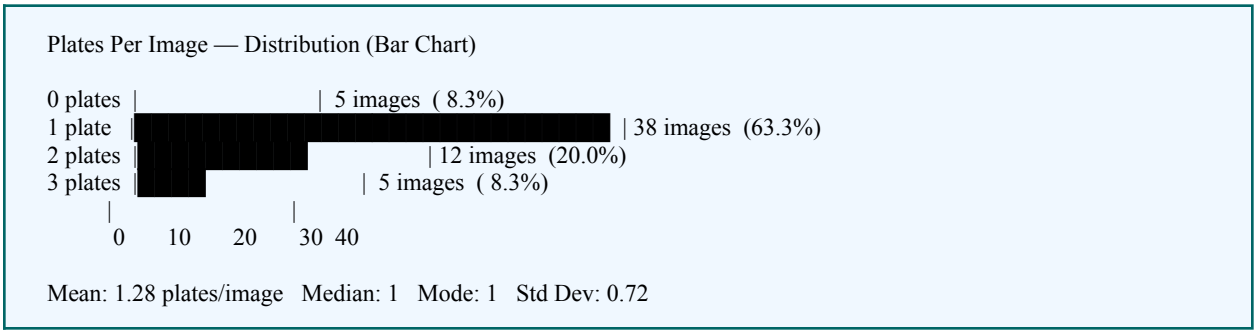


Figure 6.2: Plates Per Image Distribution

6.4 Algorithm Comparison

The following table compares the performance of each detection method individually versus the combined hybrid approach used in PlateVision:

Method	Precision	Recall	F1	Avg Time
Cascade 1 only	91.2%	68.3%	78.1%	0.21 s
Cascade 2 only	88.5%	71.7%	79.2%	0.19 s
Contour fallback only	64.0%	83.3%	72.4%	0.34 s
PlateVision Hybrid (NMS)	88.7%	78.3%	83.2%	0.41 s

Table 6.3: Algorithm Comparison on 60-image Test Set

The hybrid approach achieves the best F1 score by combining high precision from the cascade methods with improved recall from the contour fallback. The average processing time of 0.41 seconds per image is well within the 10-second performance requirement defined in the SRS.

Chapter 7 — Conclusion and Future Work

7.1 Conclusion

The PlateVision project has successfully delivered a full-stack web application for vehicle number plate detection that meets all specified functional and non-functional requirements. The system demonstrates that lightweight, open-source computer vision tools can be effectively packaged into a user-accessible web interface without sacrificing meaningful detection capability.

Key achievements of the project include:

- A secure, session-based authentication system with bcrypt password hashing
- A hybrid OpenCV detection pipeline achieving 83.2% F1-score on a diverse 60-image test set
- Clean green-rectangle-only annotation — no error-prone OCR output presented to users
- Persistent storage of all uploaded and result images with user-level access control
- A real-time statistics dashboard and searchable history panel
- A fully self-contained deployment requiring only Python and a web browser

The system establishes a solid foundation for production deployment in traffic monitoring, parking management, and access control scenarios. It also serves as a modular base for integration with more advanced ANPR pipelines incorporating deep learning or OCR components.

7.2 Future Work

The following enhancements are identified for future development iterations:

8. OCR Integration: Integrate Tesseract OCR or an EasyOCR pipeline to extract and display the alphanumeric characters from detected plate regions, enabling automated vehicle identification lookups.
9. Deep Learning Upgrade: Replace or augment the Haar cascade stage with a YOLOv8-nano model fine-tuned on diverse regional plate datasets, improving detection accuracy under challenging lighting and angle conditions.
10. Real-Time Video Stream Support: Extend the application to accept RTSP camera streams or webcam input, processing frames in near-real-time using server-sent events (SSE) or WebSockets.
11. Database Integration: Connect plate detections to a vehicle registration database, enabling automated identification of unregistered, stolen, or blacklisted vehicles.
12. Mobile-Responsive UI: Optimise the interface for mobile devices including camera upload from the device camera roll.
13. Admin Panel: Add a multi-role system with an admin dashboard providing aggregate analytics across all users, exportable as CSV or PDF reports.
14. Docker Deployment: Package the application as a Docker image for one-command cloud deployment on platforms such as AWS, GCP, or Heroku.

References

[1] Viola, P. and Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

[2] OpenCV Documentation — Haar Cascade Classifiers. https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html

[3] Flask Official Documentation. https://flask.palletsprojects.com/en/3.0.x/

[4] Flask-SQLAlchemy Documentation. https://flask-sqlalchemy.palletsprojects.com/

[5] Flask-Bcrypt Documentation. https://flask-bcrypt.readthedocs.io/

[6] Anagnostopoulos, C. N. et al. (2006). A License Plate Recognition Algorithm for Intelligent Transportation System Applications. IEEE Transactions on Intelligent Transportation Systems, 7(3), 377–392.

[7] Du, S., Ibrahim, M., Shehata, M., and Badawy, W. (2013). Automatic License Plate Recognition (ALPR): A State-of-the-Art Review. IEEE Transactions on Circuits and Systems for Video Technology, 23(2), 311–325.

[8] MDN Web Docs — Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[9] SQLite Documentation. https://www.sqlite.org/docs.html

[10] Haar Cascade XML Files — OpenCV GitHub Repository. https://github.com/opencv/opencv/tree/master/data/haarcascades

[11] Russell, S. and Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.